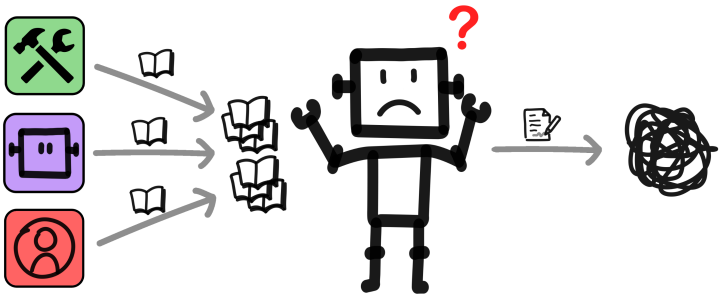
Separating Discovery from Execution: Capability Gateways for Agent Systems

Maximilian Armuss[1], Max Huppertz[1], Dong Sicheng[1], Li Penglin[1], Maximilian Wich[2], Alecio Binotto[2]

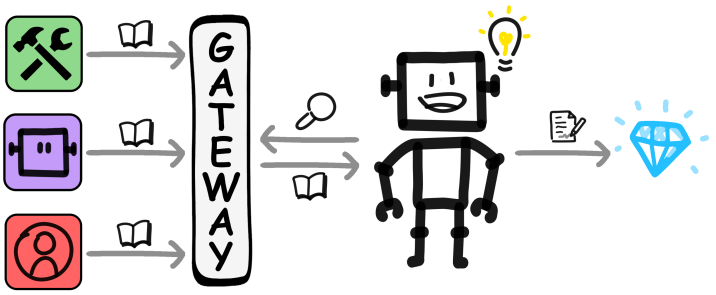
[1] Technische Universität München [2] Carl Zeiss AG

The Problem ⚠️



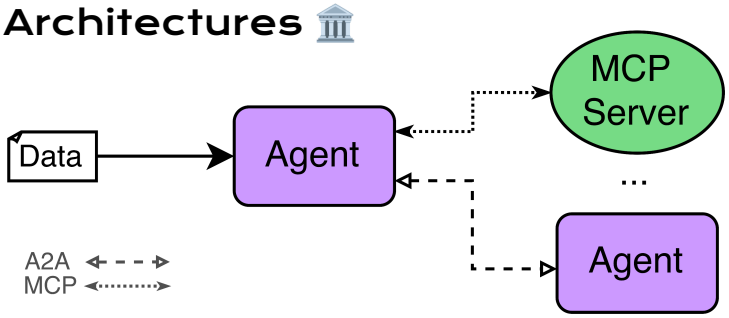
Agents are meant to be autonomous in their decision-making and data retrieval. However, multi-agent frameworks often **bloat agent context windows** by dumping raw descriptions of all available resources into the prompt. From our perspective, the key issue is that **discovery is not separated from execution**: agents receive both *what a capability does* and *how to call it* at the same time, instead of getting the right information at the right stage. As a result, planning happens in a sea of task-irrelevant details, **making correct decisions and reliable use much harder.**

The Solution 💡



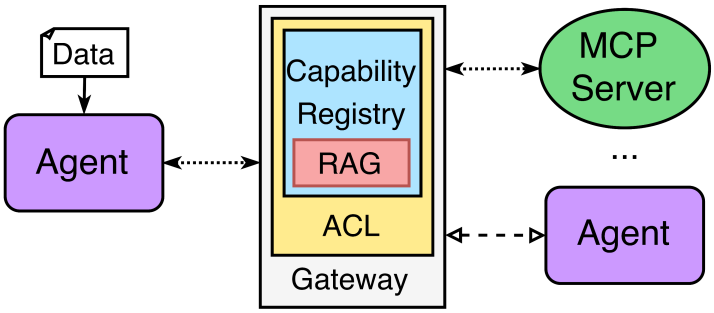
To restore full agent autonomy, we propose a unified gateway that **splits task completion into two phases**. During **discovery**, agents only see short, semantic capability summaries and retrieve the most relevant options. Our capability concept unifies peer agent skills (A2A) and tools (MCP). Once an agent commits to **execution**, it requests the required technical calling details and invokes the selected capability through the gateway. This separation **preserves mental clarity during planning without sacrificing reliable, correct tool use**.

Architectures 🏛️



We consider two core architectures, each implemented in two variants:

- In the **Simple** architecture (built with CrewAI), each agent must handle both A2A communication and MCP tooling, which frequently leads to parsing errors and tool-call confusion, so runs often do not complete (N/A).
- The **Simple-Crew** variant replaces single agents with a two-agent crew, where one agent specializes in A2A communication and the other in MCP tool use. This produces usable results, but still suffers from framework overhead.



- Our **Gateway** minimizes context bloat by exposing only three deliberately thin MCP tools (*discover* / *details* / *execute*) that provide access to *all allowed* capabilities behind the gateway. This keeps the caller agent lightweight, since it no longer has to manage direct A2A / MCP connections by itself.
- Building on this, **Gateway-RAG** turns discovery into a query-driven step. In a semantic request, the caller describes what it wants to achieve and receives only the *top-K allowed* capabilities most likely to help complete the task.

Setup & Results 📊

Below, we evaluate the four pipeline variants on the same workflow: One agent looks up a known vulnerability from an open vulnerability database, a second agent checks the software inventory list (SBOMs) for affected packages, and a third agent drafts a notification to the responsible owners. The final notification result should contain e.g. affected packages, versions, and remediation steps.
The concrete **domain is not the focus** here — it mainly provides a realistic scenario with reliable ground truth, so we can isolate **how different architectures behave under identical instructions**.

In order to compare the four architectures fairly, we use the aggregate metrics **latency**, **token consumption** (in and out), and **accuracy**. We evaluate on 20 distinct, realistic systems and report the mean across all scenarios for each metric. The accuracy of an end-to-end pipeline run is determined by the similarity between the generated outputs and the ground-truth template, where matching fields are rewarded, and missing or incorrect fields are penalized.

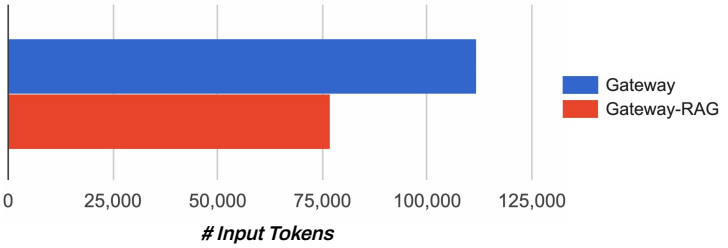
We observe that our **Gateway** architecture not only **reduces token usage**, but also **improves accuracy**. A likely explanation is that agents can spend more of their limited context budget on actual reasoning, because protocol overhead like tool-calling conventions, A2A message handling, and related boilerplate is handled by the gateway instead of being managed by each agent. We also see a **substantial latency reduction** compared to the CrewAI implementations, aligning with the absence of tool-calling loops and framework-induced handoff overhead.

	Latency	Tokens In	Tokens Out	Accuracy
Simple	N/A	N/A	N/A	N/A
Simple-Crew	853.70s	74.43k	40.13k	66.20%
Gateway	169.72s	65.69k	11.04k	**90.73%**
Gateway-RAG	**164.87s**	**64.47k**	**10.63k**	90.52%

N/A: run did not complete due to collaboration failure, etc.

Scaling Study 📝

In these baseline results, Gateway and Gateway-RAG look almost identical due to the evaluation environment containing only the required capabilities. With no task-irrelevant capabilities to filter out, semantic discovery may even perform worse.
We therefor further experiment by **enlarging the environment** with additional domain-relevant capabilities. In this larger setting, **Gateway-RAG** matches the vanilla Gateway's accuracy but uses **significantly fewer input tokens**. This gap widens as the environment grows, indicating that semantic retrieval prevents unnecessary information from entering the agent's context window.



TL:DR; ⚡

Context bloat isn't just "too many tools" — it's mixing *what a tool does* with *how to call it* during planning. Our gateway decouples discovery from execution, providing the right information at the right stage. This approach yields lower token / latency overhead and better accuracy. Semantic, query-based retrieval pays off even more in larger, tool-heavy environments.

Above QR code leads you to our GitHub repository (paper + code)!
Link: https://github.com/maximilian-armuss-dev/capability-discovery-gateway